

Contents

Getting started	1
1. Install Docker	1
Windows and macOS	1
Linux	1
2. Clone the course repository	2
3. Start the environment	2
4. Getting each new lesson	3
Troubleshooting	3
Before Lesson 9: switching to the full image	4
Running without cloning (not recommended)	4

Getting started

Everything you need for this course runs inside a Docker container: Python, JupyterLab, NumPy, pandas, scikit-learn and matplotlib are already installed. You do not install Python on your machine.

You will never need an API key or a paid account for this course.

The image comes in two sizes. You start with **core**, which is everything lessons 1 to 8 need. Before Lesson 9 you switch one line to **full**, which adds TensorFlow for the two neural-network lessons — and because **full** is built on top of **core**, that switch downloads only the part you do not already have.

1. Install Docker

You need **8 GB of RAM** and about **10 GB of free disk space** — 4 GB for the **core** image and room for the **full** one in November. That much is the same everywhere; how you install differs by platform.

Windows and macOS

Download **Docker Desktop** from docker.com and start it. That is the whole step: it installs the engine, the command line and **docker compose** together.

Linux

Docker Desktop exists for Linux, but it is not the usual route — install **Docker Engine** instead, from Docker’s own repository. The version your distribution ships is often older, and on Ubuntu it gives you **docker-compose** (with a hyphen), which is version 1 and not what this course uses.

On **Ubuntu**, add the repository:

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
-o /etc/apt/keyrings/docker.asc
```

```
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) \
signed-by=/etc/apt/keyrings/docker.asc] \
https://download.docker.com/linux/ubuntu \
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" \
| sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

then install the engine, the command line and the compose plugin:

```
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
```

On **Debian**, the same commands work with **ubuntu** changed to **debian** in both the key URL and the repository line. On **Fedora** and other Red Hat derivatives the equivalent is at docs.docker.com/engine/install.

Then do the step everyone forgets. Add yourself to the **docker** group, or every command in this document needs **sudo** in front of it:

```
sudo usermod -aG docker $USER
```

Log out and back in for that to take effect — opening a new terminal window is not enough, and neither is `su - $USER`.

Finally, check what you have:

```
docker compose version
```

It must print **v2** or later. If it prints `docker-compose version 1.x` you are looking at the old Python tool: this course uses `docker compose`, with a space.

2. Clone the course repository

```
git clone https://github.com/fabioantonini/technologies-for-artificial-intelligence.git
cd technologies-for-artificial-intelligence
```

This is how you will receive new lessons throughout the course, so clone it somewhere you will find again — not your Downloads folder.

3. Start the environment

From inside the repository folder:

```
docker compose pull
```

Do this at home, before the first lecture. It is about **750 MB** over the network, which unpacks to roughly 3 GB on disk. That is a few minutes on a home connection and a bad afternoon if thirty of us start it at once on the lecture-theatre wifi. Then start it:

```
docker compose up
```

Later runs start in seconds and download nothing.

Then open:

```
http://127.0.0.1:8888/lab?token=aicourse
```

JupyterLab opens with the course material already there. Anything you save is written to your own machine, inside the folder you cloned.

To stop it, press **Ctrl+C** in the terminal, or run `docker compose down`.

4. Getting each new lesson

New material is published on the day of each lesson. To fetch it:

```
git pull
```

That is all. A few megabytes, a couple of seconds — **you do not re-download the Docker image**. The image only changes when the software environment does, which is rare, and you will be told when it happens.

If you have edited a course notebook and `git pull` complains about a conflict, the simplest fix is to copy your version to a new name (`my_lesson3.ipynb`), then run `git checkout -- .` and pull again. Better still: work on copies from the start, and keep your own work in a `my_work/` folder.

Troubleshooting

Port 8888 is already in use. Something else is using it. Either stop that, or edit `docker-compose.yml` and change "8888:8888" to "8889:8888", then open `http://127.0.0.1:8889/lab?token=aicourse`.

“Permission denied” when saving a notebook. This should not happen — the compose file is configured to prevent it. If it does, stop the container, run `docker compose down`, then `docker compose up` again. Report it if it persists.

The container stops at once, after “Ensuring /home/jovyan is owned” (macOS). You see a list of `chown: ... Permission denied` lines on files under `.git`, then exited with code 1, and nothing answers on port 8888. This was a fault in the course’s compose file, fixed on 10 October 2026: run `git pull`, then `docker compose pull` and `docker compose up` again.

Wait for the address before opening the browser. JupyterLab is ready only when the terminal prints a line containing `http://127.0.0.1:8888/lab?token=`; before that, the browser finds nothing listening.

The container is slow to start the first time. Expected: the image is initialising and the environment is large. Subsequent starts are much faster.

I deleted my container, is my work gone? No. Your files live in the folder you cloned, on your own machine. Containers are disposable; your work is not.

Do I need a GPU? No. Every lab in this course runs on CPU by design.

Everything in my clone is suddenly owned by root (Linux). You ran `docker compose up` under `sudo` before joining the `docker` group, so the container wrote its files as `root` and `git pull` now fails. From inside the repository folder, `sudo chown -R $USER:$USER .` repairs it; joining the group, as in step 1, stops it happening again.

Before Lesson 9: switching to the full image

Lessons 9 and 10 use TensorFlow, which is large and which nothing before them needs. It is therefore not in the image you have been using. To add it, open the `.env` file in the repository folder and change one line:

```
TAI_TAG=full
```

Then, at home and before the lecture:

```
docker compose pull
docker compose up
```

You are not downloading the environment again. Everything except TensorFlow is already on your machine, so only that layer is fetched.

Running without cloning (not recommended)

The published image contains a snapshot of the course, so this works:

```
docker run --rm -p 8888:8888 -e JUPYTER_TOKEN=aicourse \
  fabioantonini/technologies-for-artificial-intelligence:core
```

But your work is stored **inside the container and lost when it stops**, and the snapshot inside the image is frozen at the moment it was built, so you would not receive new lessons at all. Use the `git clone` route above.